

JavaScript Event Listener

[Back to JS page](#)

Table of Contents

- [JavaScript Event Listener](#)
 - [The addEventListener\(\) Method](#)
 - [Example 1](#)
 - [Add an Event Handler to an Element](#)
 - [Example 2](#)
 - [Add Many Event Handlers to the Same Element](#)
 - [Example 3](#)
 - [Add an Event Handler to the window Object](#)
 - [Example 4](#)
 - [Passing Parameters](#)
 - [Example 5](#)
 - [Event Bubbling or Event Capturing](#)
 - [Example 6](#)
 - [The removeEventListener\(\) Method](#)
 - [Example 7](#)
 - [Document](#)
 - [Reference](#)

The addEventListener() Method

The `addEventListener()` method attaches an event handler to the specified element.

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element.

You can add many event handlers of the same type to one element, i.e two "click" events.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The addEventListener() Method</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").innerHTML = "Hello World!";
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Add an Event Handler to an Element

You can add an event handler to any DOM element using `addEventListener()` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add an Event Handler to an Element</h4>
<p id="demo">Click the button to change text color.</p>
<button id="myBtn">Click me</button>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").style.color = "red";
});
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Add Many Event Handlers to the Same Element

The `addEventListener()` method allows you to add many events to the same element, without overwriting existing events:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add Many Event Handlers</h4>
<button id="myBtn">Try it</button>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = document.getElementById("myBtn");
x.addEventListener("click", function1);
x.addEventListener("click", function2);

function function1() {
    document.getElementById("demo1").innerHTML = "First function executed!";
}

function function2() {
    document.getElementById("demo2").innerHTML = "Second function executed!";
}
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Add an Event Handler to the window Object

The `addEventListener()` method allows you to add event listeners to any HTML DOM object, such as HTML elements, the HTML document, the window object, or other objects that support events, like the `xmlHttpRequest` object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Add an Event Handler to the window Object</h4>
<p id="demo"></p>

<script>
window.addEventListener("resize", function() {
    document.getElementById("demo").innerHTML = "Window resized! Width: " + window.innerWidth;
});
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Passing Parameters

When passing parameter values, use an anonymous function that calls the specified function with the parameters:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Passing Parameters</h4>
<button id="myBtn">Try it</button>
<p id="demo"></p>

<script>
let p1 = 5;
let p2 = 7;
document.getElementById("myBtn").addEventListener("click", function() {
    myFunction(p1, p2);
});

function myFunction(a, b) {
    document.getElementById("demo").innerHTML = "Result: " + (a * b);
}
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Event Bubbling or Event Capturing

There are two ways of event propagation in the HTML DOM: **bubbling** and **capturing**.

In **bubbling**, the inner most element's event is handled first and then the outer.

In **capturing**, the outer most element's event is handled first and then the inner.

With `addEventListener()`, you can specify the propagation type by using the `useCapture` parameter:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Event Capturing</h4>
<div id="myDiv" style="background-color: coral; padding: 40px;">
  <button id="myBtn">Click me</button>
</div>
<p id="demo"></p>

<script>
document.getElementById("myDiv").addEventListener("click", function() {
  document.getElementById("demo").innerHTML += "DIV clicked (capturing)<br>";
}, true);

document.getElementById("myBtn").addEventListener("click", function() {
  document.getElementById("demo").innerHTML += "BUTTON clicked<br>";
}, true);
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

The removeEventListener() Method

The `removeEventListener()` method removes event handlers that have been attached with the `addEventListener()` method:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Listener</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The removeEventListener() Method</h4>
<button id="myBtn">Click me</button>
<button id="removeBtn">Remove event</button>
<p id="demo"></p>

<script>
function myFunction() {
    document.getElementById("demo").innerHTML += "Button clicked!<br>";
}

document.getElementById("myBtn").addEventListener("click", myFunction);

document.getElementById("removeBtn").addEventListener("click", function() {
    document.getElementById("myBtn").removeEventListener("click", myFunction);
    document.getElementById("demo").innerHTML = "Event handler removed!";
});
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Event Listener](#)